DSA Project Report

Cash Flow Minimizer

Muhammad Fahaz Khan

Table of Contents

1. **Introduction** 3
2. **Problem Statement** 3
3. **Objectives** 3
4. **Literature Review** 3
5. **Data Structures and Algorithms Used** 3
6. **Proposed Solution** 4
7. **System Design and GUI Overview** 4
8. **Implementation Details** 4
9. **Code Snippet** 4
10. **Output Screenshot** 9
11. **Conclusion and Future Enhancements** 9

Muhammad Fahaz Khan

1. Introduction

Managing transactions within a group of people becomes complex when multiple debts are involved. Our project, Cash Flow Minimizer, is designed to reduce the number of transactions required to settle all group debts by using an efficient algorithmic approach. The application is implemented in Java with a Swing GUI interface, making it user-friendly and interactive.

2. Problem Statement

When several people owe and lend money within a group, direct settlements can lead to a high number of redundant transactions. This results in unnecessary effort and confusion. The challenge is to minimize these transactions while maintaining accurate settlements for everyone involved.

3. Objectives

- Reduce the total number of transactions required to settle group debts.
- Implement a greedy algorithm for efficient debt simplification.
- Create a user-friendly Java Swing GUI for interaction.

4. Literature Review

Traditional methods rely on manual tracking or spreadsheets, which are inefficient for large groups. Algorithmic methods, especially using graph theory and greedy algorithms, are well-suited for this optimization problem. Our approach is inspired by network flow optimization and smart pairing using heaps.

5. Data Structures and Algorithms Used

- HashMap: Stores net balances of each user (name as key, balance as value).
- PriorityQueue: Used as a min-heap and max-heap to sort debtors and creditors.
- ArrayList: Stores the initial transactions.
- Greedy Algorithm: Matches the largest debtor with the largest creditor at every step to minimize transaction count.

Muhammad Fahaz Khan

6. Proposed Solution

- Accept transactions as input from the user.

- Calculate net balances using a HashMap.

- Separate creditors and debtors into PriorityQueues.

- Use a greedy approach to minimize transactions: always match the person who owes the most with the one owed the most.

- Display minimized results to the user.

7. System Design and GUI Overview

The application consists of the following GUI screens:

- Home Page: Contains navigation buttons for 'Enter Transactions', 'How It Works', and 'Exit'.

- How It Works Page: Describes the concept of the Cash Flow Minimizer.

- Transaction Page: Allows input of transactions and displays both the original and minimized lists.

8. Implementation Details

- The application is built in Java using Swing.

- The GUI files are modular:

  - HomePage.java: Entry point to the app

  - HowItWorksPage.java: Displays theory and explanation

  - CashFlowMinimizerGUI.java: Main logic for transaction entry and minimization

9. Code Snippet

```
import javax.swing.*;

import java.awt.*;
import java.util.*;

public class CashFlowMinimizerGUI extends JFrame {
```

Muhammad Fahaz Khan

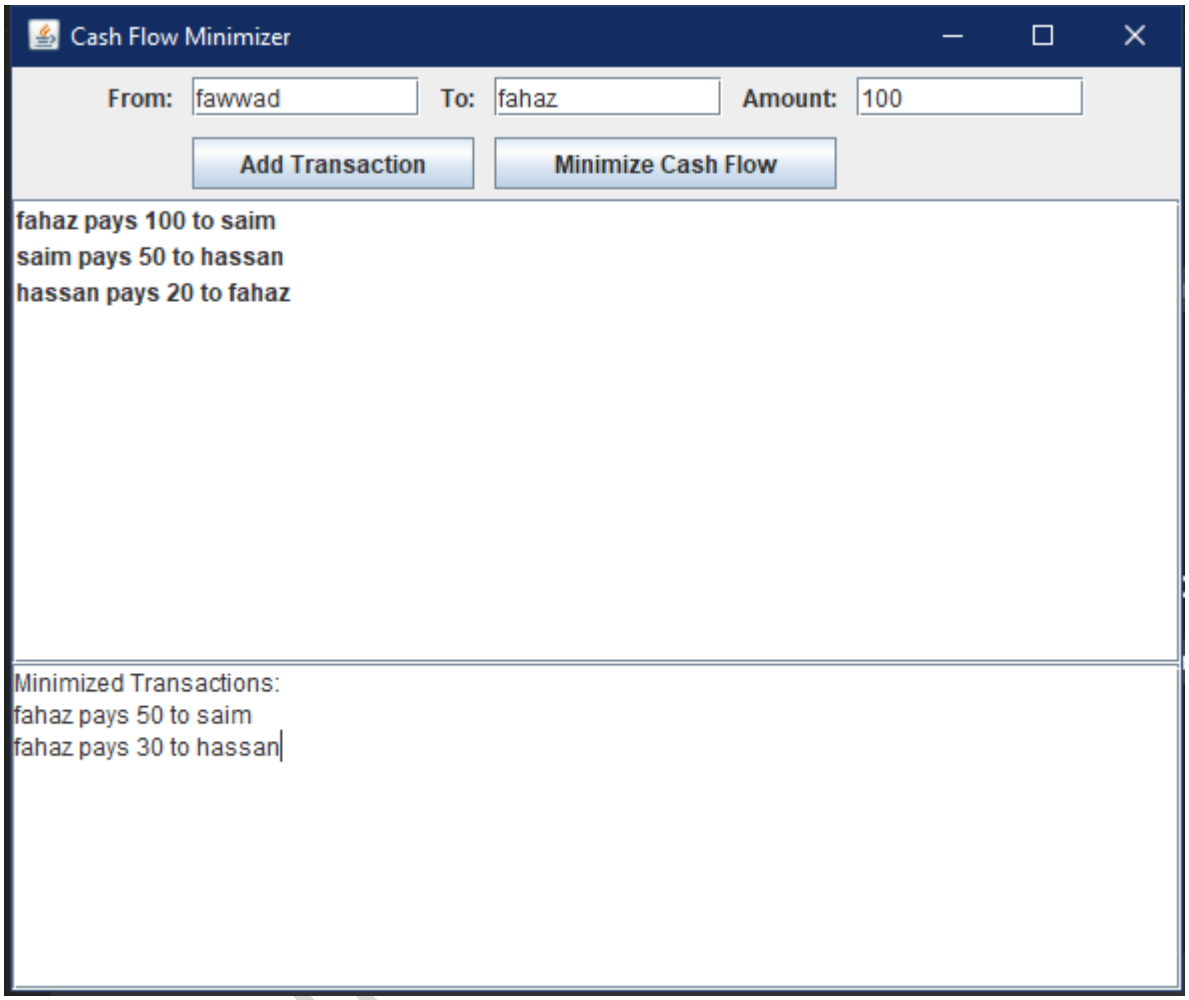
```java
    private DefaultListModel<String> transactionListModel = new
DefaultListModel<>();
    private java.util.List<Transaction> transactions = new ArrayList<>();

    private JTextField fromField = new JTextField(10);
    private JTextField toField = new JTextField(10);
    private JTextField amountField = new JTextField(10);
    private JTextArea outputArea = new JTextArea(10, 30);

    public CashFlowMinimizerGUI() {
        setTitle("Cash Flow Minimizer");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(600, 500);
        setLocationRelativeTo(null);

        // ==== INPUT PANEL WITH GRIDBAGLAYOUT ====
        JPanel inputPanel = new JPanel(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(5, 5, 5, 5);  // Padding around each component
        gbc.fill = GridBagConstraints.HORIZONTAL;

        // Row 1: Labels and Fields
        gbc.gridx = 0;
        gbc.gridy = 0;
        inputPanel.add(new JLabel("From:"), gbc);

        gbc.gridx = 1;
        inputPanel.add(fromField, gbc);

        gbc.gridx = 2;
        inputPanel.add(new JLabel("To:"), gbc);

        gbc.gridx = 3;
        inputPanel.add(toField, gbc);

        gbc.gridx = 4;
        inputPanel.add(new JLabel("Amount:"), gbc);

        gbc.gridx = 5;
        inputPanel.add(amountField, gbc);

        // Row 2: Buttons
        JButton addButton = new JButton("Add Transaction");
        JButton minimizeButton = new JButton("Minimize Cash Flow");
```

```
        gbc.gridx = 1;
        gbc.gridy = 1;
        gbc.gridwidth = 2;
        inputPanel.add(addButton, gbc);

        gbc.gridx = 3;
        gbc.gridwidth = 2;
        inputPanel.add(minimizeButton, gbc);

        // ==== TRANSACTION LIST + OUTPUT AREA ====
        JList<String> transactionList = new JList<>(transactionListModel);
        JScrollPane transactionScroll = new JScrollPane(transactionList);
        JScrollPane outputScroll = new JScrollPane(outputArea);
        outputArea.setEditable(false);

        // ==== LAYOUT ====
        setLayout(new BorderLayout());
        add(inputPanel, BorderLayout.NORTH);
        add(transactionScroll, BorderLayout.CENTER);
        add(outputScroll, BorderLayout.SOUTH);

        // ==== BUTTON ACTIONS ====

        // Add Transaction
        addButton.addActionListener(e -> {
            try {
                String from = fromField.getText().trim();
                String to = toField.getText().trim();
                int amount = Integer.parseInt(amountField.getText().trim());

                if (from.equals("") || to.equals("") || from.equals(to)) {
                    JOptionPane.showMessageDialog(this, "Invalid input!");
                    return;
                }

                transactions.add(new Transaction(from, to, amount));
                transactionListModel.addElement(from + " pays " + amount + " to "
+ to);

                fromField.setText("");
                toField.setText("");
                amountField.setText("");
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(this, "Amount must be a number!");
            }
```

```java
        });

        // Minimize Cash Flow
        minimizeButton.addActionListener(e -> {
            Map<String, Integer> net = calculateNetAmounts(transactions);
            java.util.List<String> result = minimizeCashFlow(net);
            outputArea.setText("Minimized Transactions:\n");
            for (String s : result) {
                outputArea.append(s + "\n");
            }
        });
    }

    // ==== CORE LOGIC ====

    private Map<String, Integer> calculateNetAmounts(java.util.List<Transaction>
transactions) {
        Map<String, Integer> netAmounts = new HashMap<>();
        for (Transaction t : transactions) {
            netAmounts.put(t.from, netAmounts.getOrDefault(t.from, 0) -
t.amount);
            netAmounts.put(t.to, netAmounts.getOrDefault(t.to, 0) + t.amount);
        }
        return netAmounts;
    }

    private java.util.List<String> minimizeCashFlow(Map<String, Integer>
netAmounts) {
        PriorityQueue<Person> creditors = new PriorityQueue<>((a, b) -> b.amount
- a.amount);
        PriorityQueue<Person> debtors = new
PriorityQueue<>(Comparator.comparingInt(a -> a.amount));
        java.util.List<String> result = new ArrayList<>();

        for (Map.Entry<String, Integer> entry : netAmounts.entrySet()) {
            int amt = entry.getValue();
            if (amt > 0)
                creditors.add(new Person(entry.getKey(), amt));
            else if (amt < 0)
                debtors.add(new Person(entry.getKey(), amt));
        }

        while (!creditors.isEmpty() && !debtors.isEmpty()) {
            Person cr = creditors.poll();
            Person db = debtors.poll();
```

```java
            int settle = Math.min(cr.amount, -db.amount);
            result.add(db.name + " pays " + settle + " to " + cr.name);

            int newCrAmt = cr.amount - settle;
            int newDbAmt = db.amount + settle;
            if (newCrAmt > 0) creditors.add(new Person(cr.name, newCrAmt));
            if (newDbAmt < 0) debtors.add(new Person(db.name, newDbAmt));
        }
        return result;
    }

    // ==== DATA CLASSES ====

    static class Transaction {
        String from, to;
        int amount;
        public Transaction(String from, String to, int amount) {
            this.from = from;
            this.to = to;
            this.amount = amount;
        }
    }

    static class Person {
        String name;
        int amount;
        public Person(String name, int amount) {
            this.name = name;
            this.amount = amount;
        }
    }

    // ==== MAIN ====
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new
CashFlowMinimizerGUI().setVisible(true));
    }
}
```

10. Output Screenshot



11. Conclusion and Future Enhancements

The Cash Flow Minimizer successfully reduces redundant transactions using a greedy algorithm. It demonstrates efficient use of Java's data structures and GUI capabilities.

Future Improvements:

- Export minimized results to a `.txt` or `.pdf` file.
- Add chart or graphical visualizations.
- Extend to multi-currency or weighted priority-based debts.
- Add login or user history saving features.